

UPPSALA UNIVERSITY



BACHELOR'S THESIS

Martingale Matching

Authors:

Anton VON
NANDELSTADH

May 24, 2026

Contents

1	Introduction	2
2	Background	2
2.1	SDE Preliminaries	2
2.2	Generative Models	10
3	Algorithm	13
3.1	Training	13
3.2	Generation	17
4	Results	19
5	Conclusion	23

1 Introduction

This thesis aims to develop a novel method for learning the stochastic differential equation of a diffusion-based generative model, which we call martingale matching. We present this as an alternative to the predominant frameworks of score matching [13] and flow matching [8].

The first diffusion models were discrete-time Gaussian processes [11] [2]. These were later reformulated as stochastic differential equations (SDEs) [13]. In a diffusion model, the probability path interpolating between a simple distribution and the data distribution is created through a forward noising process modeled by an SDE. Inspired by the reversal of the forward noising process [1], these SDEs are chosen such that they have marginal probabilities which can parametrize the generator of the diffusion process using the score function, hence the name score matching [12] [9]. Our discussions regarding the framework of flow/score matching mostly follow the conventions used by [9], [3]. Specifically, we consider the SDE we want to learn as moving forward in time, unlike the theory of time-reversal used in the older score matching literature, such as [13].

Our novel approach is to learn the generator of the diffusion process directly, instead of using the score to do this implicitly. We do this by enforcing a known martingale from the basic theory of SDEs. We then use deviations from the martingale to construct a new loss function, based on which we train and learn the drift and diffusion functions. We present our results applying this new methodology to various toy datasets, and discuss the possibility of using it to train on high-dimensional data. In order to do this, we first develop some of the theory of stochastic differential equations, and the general theory of diffusion-based generative models.

2 Background

2.1 SDE Preliminaries

In order to understand denoising diffusion models, we first need to go through some preliminaries regarding stochastic processes and stochastic differential equations. In what follows, we assume basic knowledge of measure theory and measure-theoretic probability theory. We begin by expanding our regular probability space, defined by a triple, into a filtered probability space, defined

by a quadruple.

Definition 2.1 (Filtration)

Let $(\Omega, \mathcal{F}, P)$ be a probability space and let T be an index set. For each $i \in T$, let $\mathcal{F}_i$ be a sub- σ -algebra of $\mathcal{F}$. Then $\{\mathcal{F}_i\}_{i \in T}$ is a filtration if $\mathcal{F}_k \subset \mathcal{F}_l$ for each $k \leq l$. Then $(\Omega, \mathcal{F}, \{\mathcal{F}_t\}, P)$ is called a filtered probability space.

We are now ready for one of the central definitions of modern probability theory, that of a martingale.

Definition 2.2 (Martingale)

A stochastic process $X : T \times \Omega \rightarrow \mathbb{R}$ on probability space $(\Omega, \mathcal{F}, P)$ is a martingale w.r.t. the filtration $\{\mathcal{F}_t\}$ (of $(\Omega, \mathcal{F}, P)$) if

1. X is adapted to the filtration. That is, X_t is $\mathcal{F}_t$ measurable for each $t \in T$.
2. for each t , $\mathbb{E}[X_t] < \infty$
3. for all s and t with $s < t$, $\mathbb{E}[X_t | \mathcal{F}_s] = X_s$.

We intend to construct the Itô integral, which will be essential in our definition of stochastic differential equations. The exposition of the construction follows [10]. We begin by defining the class ν of functions for which the Itô integral will exist.

Definition 2.3

Let $\nu = \nu(S, T)$ be the class of functions

$$f(t, \omega) : [0, \infty) \times \Omega \rightarrow \mathbb{R}$$

such that

1. $(t, \omega) \rightarrow f(t, \omega)$ is $\mathcal{B} \times \mathcal{F}$ measurable, where $\mathcal{B}$ denotes the Borel σ -algebra on $[0, \infty)$.
2. $f(t, \omega)$ is $\mathcal{F}_t$ -adapted
3. $\mathbb{E}[\int_S^T f(t, \omega)^2 dt] < \infty$

Similarly to how we constructed the Lebesgue integral in analysis from simple functions, we will define a set of elementary functions, and show that the integral of $f \in \nu$ can be defined as an L^2 limit of these elementary functions. We define an elementary functions as follows:

Definition 2.4 (Elementary function)

A function $\phi \in \nu$ is called elementary if it has the form

$$\phi(t, \omega) = \sum_j e_j(\omega) \chi_{[t_j, t_{j+1})}(t). \quad (1)$$

Now, again similarly to how we do Lebesgue integration, we define the integral of these elementary functions in the following way:

Definition 2.5 (Integral of an elementary function)

For elementary function $\phi(t, \omega)$, the integral is defined according to

$$\int_S^T \phi(t, \omega) dB_t(\omega) = \sum_{j \geq 0} e_j(\omega) [B_{t_{j+1}} - B_{t_j}](\omega). \quad (2)$$

The identity on which we will base our construction is the Itô isometry.

Definition 2.6 (Itô isometry for elementary functions)

If $\phi(t, \omega)$ is bounded and elementary, then

$$\mathbb{E} \left[\left(\int_S^T \phi(t, \omega) dB_t \right)^2 \right] = \mathbb{E} \left[\int_S^T \phi(t, \omega)^2 dt \right] \quad (3)$$

Proof. Write $\Delta B_j = B_{t_{j+1}} - B_{t_j}$. Then

$$\mathbb{E}[e_i e_j \Delta B_i \Delta B_j] = \begin{cases} 0 & \text{if } i \neq j \\ [e_j^2](t_{j+1} - t_j) & \text{if } i = j. \end{cases}$$

By independence of $e_i e_j \Delta B_i$ and ΔB_j with $i < j$, we get

$$\begin{aligned} \mathbb{E} \left[\left(\int_S^T \psi dB \right)^2 \right] &= \sum_{i,j} \mathbb{E}[e_i e_j \Delta B_i \Delta B_j] \\ &= \sum_j \mathbb{E}[e_j^2](t_{j+1} - t_j) \\ &= \mathbb{E} \left[\int_S^T \phi^2 dt \right] \end{aligned}$$

□

We are then ready to extend the definition of the integral, from elementary functions to arbitrary $f \in \nu$, using the Itô isometry. The following section follows [10] closely. For a more detailed proof, especially of the subtleties regarding measurability, see e.g. [6].

Step 1. Let $g \in \nu$ be bounded and $g(\cdot, \omega)$ continuous for each ω . Then there exist elementary functions $\phi_n \in \nu$ such that

$$\mathbb{E} \left[\int_S^T (g - \phi_n)^2 dt \right] \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

Proof. Define $\phi_n(t, \omega) = \sum_j g(t_j, \omega) \chi_{[t_j, t_{j+1})}(t)$. Then ϕ_n is elementary, and

$$\int_S^T (g - \phi_n)^2 dt \rightarrow 0 \quad \text{as } n \rightarrow \infty \text{ pointwise} \quad (4)$$

since $g(\cdot, \omega)$ is continuous for fixed ω . Hence the result follows by bounded convergence. $\square$

Step 2. Let $h \in \nu$ be bounded. Then there exist bounded functions $g_n \in \nu$ such that $g_n(\cdot, \omega)$ is continuous for all ω and n , and

$$\mathbb{E} \left[\int_S^T (h - g_n)^2 dt \right] \rightarrow 0.$$

Proof. Suppose $|h| \leq M$. For each n , let $\phi_n \geq 0$ be a continuous function on $\mathbb{R}$ such that

1. $\phi_n(x)n = 0$ for $x \leq -\frac{1}{n}$ and $x \geq 0$
2. $\int_{-\infty}^{\infty} \phi_n(x) dx = 1$

Define

$$g_n(t, \omega) = \int_0^t \phi_n(s - t) h(s, \omega) ds.$$

Then $g_n(\cdot, \omega)$ is continuous for each ω and $|g_n(t, \omega)| \leq M$. Moreover,

$$\int_S^T (g_n(s, \omega) - h(s, \omega))^2 ds \rightarrow 0 \quad \text{as } n \rightarrow \infty \quad (5)$$

By bounded convergence we get that

$$\mathbb{E} \left[\int_S^T (h(t, \omega) - g_n(t, \omega))^2 dt \right] \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

□

We skimmed over some details regarding measurability and convergence here. The interested reader is referred to [6].

Step 3. Let $f \in \nu$. Then there exists a sequence $\{h_n\} \subset \nu$ such that h_n is bounded for each n and

$$\mathbb{E} \left[\int_S^T (f - h_n)^2 dt \right] \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

Proof. Write

$$h_n(t, \omega) = \begin{cases} -n & \text{for } f(t, \omega) < -n \\ f(t, \omega) & \text{for } -n \leq f(t, \omega) \leq n \\ n & \text{for } f(t, \omega) > n. \end{cases}$$

Then the result follows by dominated convergence. □

Now by the previous steps, for $f \in \nu$ we can choose elementary functions $\phi_n \in \nu$ converging to f in L^2 norm. Hence we can define

$$\int_S^T f(t, \omega) dB_t(\omega) := \lim_{n \rightarrow \infty} \int_S^T \phi_n(t, \omega) dB_t(\omega)$$

The limit exists in L^2 since the set of integrals $\int_S^T \phi_n(t, \omega) dB_t$ forms a Cauchy sequence in L^2 by the Itô isometry. In short, we get the definition below.

Definition 2.7 (Itô integral)

Let $f \in \nu(S, T)$. Then the Itô integral of f (from S to T) is defined by

$$\int_S^T f(t, \omega) dB_t(\omega) = \lim_{n \rightarrow \infty} \int_S^T \phi_n(t, \omega) dB_t(\omega) \quad (6)$$

where $\{\phi_n\}$ is a sequence of elementary functions such that

$$\mathbb{E} \left[\int_S^T (f(t, \omega) - \phi_n(t, \omega))^2 dt \right] \rightarrow 0 \quad \text{as } n \rightarrow \infty \quad (7)$$

Now we are ready to define stochastic differential equations. Since the most important integrator (and the only one which we will use) for stochastic integrals is Brownian motion, which is nowhere differentiable, we choose to define stochastic differential equations in terms of integrals rather than derivatives.

Definition 2.8 (Stochastic differential equation)

A process X_t satisfies the stochastic differential equation

$$dX_t = b(t, X_t)dt + \sigma(t, X_t)dB_t \quad (8)$$

if it satisfies the integral equation

$$X_t = X_0 + \int_0^t b(s, X_s)ds + \int_0^t \sigma(s, X_s)dB_s \quad (9)$$

In order for an SDE to possess solutions, we must impose conditions on σ and b . The proof of the following theorem is similar to the ODE case, where the proof of existence uses Picard iteration, and the proof of uniqueness uses the Itô isometry and the Lipschitz condition. The interested reader is referred to [10] or [6].

Theorem 2.1 (Existence and uniqueness of SDEs)

Let $T > 0$ and $b : [0, T] \times \mathbb{R}^n \rightarrow \mathbb{R}^n, \sigma : [0, T] \times \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ be measurable functions satisfying

$$|b(t, x)| + |\sigma(t, x)| \leq C(1 + |x|) \quad x \in \mathbb{R}^n, t \in [0, T] \quad (10)$$

for some constant C , $|\sigma|^2 = \sum |\sigma_{ij}|^2$, and such that

$$|b(t, x) - b(t, y)| + |\sigma(t, x) - \sigma(t, y)| \leq D|x - y| \quad x, y \in \mathbb{R}^n, t \in [0, T] \quad (11)$$

for some constant D . Let Z be a random variable which is independent of the σ -algebra F_∞ generated by $B_s, s \geq 0$, and such that

$$\mathbb{E}[|Z|^2] < \infty. \quad (12)$$

Then the stochastic differential equation

$$dX_t = b(t, X_t)dt + \sigma(t, X_t)dB_t, \quad 0 \leq t \leq T, X_0 = Z \quad (13)$$

has unique continuous (in t) solution X_t with the property that X_t is adapted to the filtration $\mathcal{F}_t^Z$ generated by Z and $B_s, s \leq t$ and

$$\mathbb{E} \left[\int_0^T |X_t|^2 dt \right] < \infty. \quad (14)$$

Now we are ready to state the central definition for our application, namely that of the Itô diffusion.

Definition 2.9 (Time-homogenous Itô diffusion)

A (time-homogenous) Itô diffusion is a stochastic process $X_t(\omega) = X(t, \omega) : [0, \infty) \times \Omega \rightarrow \mathbb{R}^n$ satisfying a stochastic differential equation of the form

$$dX_t = b(X_t)dt + \sigma(X_t)dB_t, \quad t \geq s; X_s = x$$

where B_t is m -dimensional Brownian motion and $b : \mathbb{R}^n \rightarrow \mathbb{R}^n$, $\sigma : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ satisfy

$$|b(x) - b(y)| + |\sigma(x) - \sigma(y)| \leq D|x - y|; \quad x, y \in \mathbb{R}^n$$

We call b the drift coefficient and σ the diffusion coefficient.

Note that the assumptions of theorem 2.1 simplify in the case of diffusion. A very powerful result about Itô processes which we shall make use of is Itô's lemma. This is the stochastic equivalent of the chain rule in deterministic calculus.

Theorem 2.2 (Itô's lemma)

Let

$$dX(t) = udt + vB(t)$$

be an n -dimensional Itô process. Let $f(t, x)$ be a C^2 map from $[0, \infty) \times \mathbb{R}^n$ into $\mathbb{R}$. Then the process

$$Y(t, \omega) = f(t, X(t))$$

is again an Itô process, given by

$$dY = \frac{\partial f}{\partial t}(t, X)dt + \sum_i \frac{\partial f}{\partial x_i}(t, X)dX_i + \frac{1}{2} \sum_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j}(t, X)dX_i dX_j$$

where $dB_i dB_j = \delta_{i,j}dt$, $dB_i dt = dt dB_i = dt dt = 0$.

For a proof of Itô's lemma, see e.g. [10]. Another useful tool for us is the generator of the Itô diffusion, which encodes a lot of useful information about the diffusion process. Our algorithm learns the generator rather than learning the process directly.

Definition 2.10 (Infintessimal generator)

Let $\{X_t\}$ be an Itô diffusion in $\mathbb{R}^n$. The infintessimal generator $\mathcal{L}$ of X_t is defined by

$$\mathcal{L}f(x) = \lim_{t \downarrow 0} \frac{\mathbb{E}^x[f(X_t)] - f(x)}{t}; \quad x \in \mathbb{R}^n$$

for the set of functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ where the limit exists uniformly.

The generator exists for a wider class of processes than just Itô diffusion. However, in the case of diffusion, we can relate the generator to the drift and diffusion coefficients in the following way.

Theorem 2.3

The generator of the diffusion process is given by

$$\mathcal{L}_s f = b \cdot \nabla f + \frac{1}{2} \text{tr}(\sigma^T \nabla^2 f)$$

which in the case where σ is a constant simplifies to

$$\mathcal{L}_s f = b \cdot \nabla f + \frac{1}{2} \sigma^2 \Delta f$$

Here f is any C^2 function.

Proof. The Itô formula gives

$$df(X_t) = \sum_i b_i \frac{\partial f}{\partial x_i} dt + \frac{1}{2} \sum_{i,j} (\sigma \sigma^T)_{i,j} \frac{\partial^2 f}{\partial x_i \partial x_j} dt + M_t$$

where M_t is some martingale term. We rewrite this as

$$f(X_t) = f(x) + \int_0^t \mathcal{L}f(X_s) ds + M_t$$

and we now wish to show that $\mathcal{L}f$ is the generator. Taking expectations we have

$$T_t f(x) = f(x) + \int_0^t \mathbb{E}_x[\mathcal{L}f(X_s)] ds + \mathbb{E}[M_t]$$

so that

$$T_t f(x) - f(x) = \int_0^t T_s(\mathcal{L}f)(x) ds$$

which gives us that

$$T_t \frac{d}{dt} f(x) \Big|_{t=0} = \mathcal{L}f(x)$$

and so it is clear that $\|\frac{d}{dt}T_t f - \mathcal{L}f\|$ as $t \rightarrow 0$. $\square$

2.2 Generative Models

A generative model allows us to interpolate between p_{simple} and p_{data} . As the name suggests, we choose p_{simple} to be some known distribution, so that we can easily generate new samples of p_{data} by sampling $z \sim p_{simple}$ and applying the model.

Definition 2.11 (Generative modelling)

Generative modelling is the process of training a model to transform a sample from a simple distribution p_{simple} to a data distribution p_{data} .

In general, the distribution p_{data} will be unknown to us, since otherwise there would not be much to learn. Instead, we assume access to a finite number of examples of p_{data} , sampled independently, which serves to represent the true distribution. We call this our dataset.

Definition 2.12 (Dataset)

A dataset is a finite number of samples $z_1, \dots, z_N \sim p_{data}$.

Our dataset might be a collection of images, such as MNIST, or a collection of videos from YouTube. Our aim is then to train a model to generate more images of hand-written integers or YouTube videos. We shall also need the following definition:

Definition 2.13 (Probability path)

A probability path is a time-dependent probability $\{p_t\}_{0 \leq t \leq 1}$.

An increasingly common class of generative models used are flow models. Here, one learns a vector field b_t^θ , which parametrizes the ODE that carries values from p_{init} to p_{data} [9]. Having learned b_t^θ , one can then interpolate between the distributions simply using an ODE solver.

Definition 2.14 (Flow model)

A flow model is given by:

$$\begin{aligned}\frac{d}{dt}X_t &= b_t^\theta(X_t) \\ X_0 &\sim p_{init}\end{aligned}$$

where $b_t^\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is a vector field parametrized by a neural network with parameters θ .

The way flow models are trained is called flow matching, where we construct a target vector field $u_t^{target}(x)$ and minimize the loss

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}[\|u_t^\theta(x) - u_t^{target}(x)\|^2]$$

through stochastic gradient descent. One can construct the target as

$$u_t^{target}(x) = \int u_t^{target}(x|z) \frac{p_t(x|z)p_{data}(z)}{p_t(x)} dz$$

and then it holds that

$$X_0 \sim p_{simple}, \quad \frac{d}{dt}X_t = u_t^{target}(X_t) \implies X_t \sim p_t \quad (0 \leq t \leq 1)$$

[9] [4]. Unfortunately this integral is intractable, but with some more work, one can then show that the conditional flow matching loss

$$\mathcal{L}_{CFM}(\theta) = \mathbb{E}[\|u_t^\theta(x) - u_t^{target}(x|z)\|^2]$$

is the same as $\mathcal{L}_{FM}(\theta)$, up to a constant, and so minimizing one with gradient descent is equivalent to minimizing the other [9], since their gradients coincide.

Flow models were preceded by (denoising) diffusion models, which, in addition to the vector field, also have a stochastic component. The aim of classic score matching, as well as our martingale matching, is to learn the parametrized drift b_t^θ . The diffusion coefficient σ_t is chosen by us. Having learned the parametrized drift, we can generate new samples of our data distribution p_{data} , using one of many numerical SDE solvers.

Definition 2.15 (Diffusion model)

A diffusion model is given by:

$$\begin{aligned} dX_t &= b_t^\theta(X_t)dt + \sigma_t dW_t \\ X_0 &\sim p_{\text{simple}} \end{aligned}$$

where $b_t^\theta : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the parametrized drift, $\sigma_t : \mathbb{R}^d \rightarrow \mathbb{R}$ is our diffusion coefficient at time t , and p_{simple} is our initial data distribution.

The most common way to train diffusion models is through score matching [13]. The presentation of score matching given here follows [4], [9] and is different, but equivalent to, the presentation given by [13]. They prove the following theorem

Theorem 2.4

Define the conditional and marginal vector fields $u_t^{\text{target}}(x|z)$ and $u_t^{\text{target}}(x)$ as above. Then, for diffusion coefficient $\sigma_t \geq 0$, we may construct an SDE which follows the same probability path, i.e.

$$X_0 \sim p_{\text{simple}}, \quad dX_t = \left(u_t^{\text{target}}(X_t) + \frac{\sigma_t^2}{2} \nabla \log p_t(X_t) \right) dt + \sigma_t dB_t \quad (15)$$

$$\implies X_t \sim p_t \quad (0 \leq t \leq 1) \quad (16)$$

In particular, $X_t \sim p_{\text{data}}$, and this holds if we replace the marginal probability path and vector field with their conditional versions.

Now $\nabla \log p_t(x)$ is called the score, and we can write it as

$$\nabla \log p_t(x) = \int \nabla \log p_t(x|z) \frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} dz.$$

Similarly to flow matching, we approximate the score by a neural net s_t^θ and we can derive the losses

$$\mathcal{L}_{SM} = \mathbb{E}[\|s_t^\theta(x) - \nabla \log p_t(x)\|^2]$$

and

$$\mathcal{L}_{CSM} = \mathbb{E}[\|s_t^\theta(x) - \nabla \log p_t(x|z)\|^2].$$

Once again, $\mathcal{L}_{SM}$ is intractable, but we are able to show that its gradient coincides with $\mathcal{L}_{CSM}$.

Additionally, for Gaussian probability paths we can recover the vector field from the score or vice versa. This is why the traditional denoising diffusion models can be trained by only learning the score, since they are constructed based on Gaussian conditional probability paths [13].

Our algorithm, martingale matching, also learns the diffusion model, but using a completely different methodology than score matching. Instead of learning the vector field or score directly, we learn the generator of the process by enforcing a known martingale.

3 Algorithm

3.1 Training

Instead of constructing a loss using the marginal vector field, as in flow matching, or using the score, as in score matching, the idea of martingale matching is to enforce a martingale and thereby to learn the generator of the process. As we saw previously, knowledge of the generator combined with knowledge of the diffusion coefficient is enough to know the drift.

Theorem 3.1

M_t^f is a martingale where

$$M_t^f = f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s f(X_s) ds \quad (17)$$

so that $\mathbb{E}[M_t^f] = 0$. Here $f \in C^2$ is an arbitrary test function. $\mathcal{L}_s$ is the generator of the diffusion process.

Proof. This follows from the proof of theorem 2.3. $\square$

Now, we are able to construct a loss function and learn the generator by enforcing that M_t^f be a martingale. Since we already know f and σ , learning the generator means we also learn the drift term b_t^θ , as desired.

In practice, what we use is that we should have $\mathbb{E}[M_t^f] = 0$, and use deviations from this as our loss function. Thus, enforcing zero-mean allows us to learn the drift. This is how we construct our loss function.

For each sample of p_{simple} and each time step, we get residual

$$R_{t_k}^{f,i} = f(X_{t_{k+1}}^i) - f(X_{t_k}^i) - \int_{t_k}^{t_{k+1}} \mathcal{L}_s f(X_s^i) ds$$

Where t_k is the time, f is the test function, and i is the batch. We use quadrature $\int_{t_k}^{t_{k+1}} \mathcal{L}_s f(X_s) ds \approx \Delta t_k \mathcal{L}_{t_k} f(X_{t_k})$, with Δt small, to discretise the residuals and enable us to compute them efficiently. This gives discrete residual

$$R_{t_k}^{f,i} = f(X_{t_{k+1}}^i) - f(X_{t_k}^i) - \Delta t \mathcal{L}_s f(X_{t_k}^i)$$

Our loss function is then simply the mean squared of these residuals. By naively simulating the SDEs at each iteration of the algorithm, we would get a loss function like equation 18

$$L_{ME}^{naive} = \frac{1}{TN|\mathcal{F}|} \sum_{k=1}^T \sum_{f \in \mathcal{F}} \sum_{i=1}^N (R_{t_k}^{f,i})^2. \quad (18)$$

However, we instead use a Monte-Carlo approach, as in equation 19, which significantly speeds up training by each summand being conditioned on a different variable from p_{simple}

$$L_{ME} = \frac{1}{N|\mathcal{F}|} \sum_{f \in \mathcal{F}} \sum_{i=1}^N (R_{t_k}^{f,i})^2. \quad (19)$$

It remains to formally show that minimizing this error causes the resulting SDE to interpolate between p_{simple} and p_{data} .

Theorem 3.2

Minimizing equation 19 learns the desired generator.

Proof. Let $\mathcal{L}_s$ be the desired generator of our process, and $\mathcal{L}_s^\theta$ be the parametrized version based on our parametrized drift, b^θ . Then we know that

$$\mathbb{E}[f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s^\theta f(X_s) ds] = 0 \quad (20)$$

while we are trying to minimize

$$\mathbb{E}[f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s f(X_s) ds] \quad (21)$$

so the error is given by

$$\mathbb{E}[\int_0^t (\mathcal{L}_s^\theta f(X_s) - \mathcal{L}_s f(X_s) ds)]. \quad (22)$$

Then

$$\begin{aligned}
& \mathbb{E}[(f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s^\theta f(X_s) ds)^2] = \\
& \mathbb{E}[(f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s^f(X_s) ds)^2] \\
& + \mathbb{E}[f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s f(X_s) ds] \mathbb{E}[f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s^\theta f(X_s) ds] \\
& + \mathbb{E}[(\int_0^t \mathcal{L}_s f(X_s) - \mathcal{L}_s^\theta f(X_s) ds)^2]
\end{aligned}$$

but the mixed term vanishes, so we end up with

$$\begin{aligned}
& \mathbb{E}[(f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s^\theta f(X_s) ds)^2] = \\
& \mathbb{E}[(f(X_t) - f(X_0) - \int_0^t \mathcal{L}_s^f(X_s) ds)^2] + \mathbb{E}[(\int_0^t \mathcal{L}_s f(X_s) - \mathcal{L}_s^\theta f(X_s) ds)^2]
\end{aligned}$$

where the first summand does not depend on θ . Hence, minimizing $\mathbb{E}[(f(X_t) - f(X_0) - \mathcal{L}_s^\theta f(X_s) ds)^2]$, minimizes the error between the generators as well. $\square$

With our loss constructed and proven to work, we can now create our martingale matching algorithm, which is similar in vein to other algorithms for generative modelling, such as flow and score matching. The algorithm is given in Alg. 1.

Algorithm 1 Martingale Matching

Require: Neural net b_t^θ , diffusion coefficient σ , time-step Δt , dataset of samples $z \sim p_{data}$

for each mini-batch of data **do**

Sample datum z from dataset.

Sample $t \sim U[0, 1 - \Delta t]$.

Set $x_t \sim p_t(\cdot|z)$, $x_{t+\Delta t} \sim p_{t+\Delta t}(\cdot|z)$

Compute loss $\mathcal{L}_{ME}^\theta$

Update the model parameter θ via gradient descent on $\mathcal{L}_{ME}^\theta$

end for

One of the main hyperparameters thus becomes our test function, and the choice of test functions is the major design decision in any implementation of this algorithm. We want to choose a subset of functions that are representative, but computationally not very expensive. Because this choice is rather arbitrary, many modern generative methods try to avoid it altogether [4]. One set of test functions is that of random Fourier features. These have the advantage of being computationally, particularly for higher dimensional data, as the number of functions required does not necessarily depend on the dimension of the data.

Definition 3.1 (Random Fourier features)

A random Fourier feature is a function of the type

$$f_{\omega_1}(x) = \cos(w^\top x) \quad f_{\omega_2}(x) = \sin(w^\top x)$$

where $x \in \mathbb{R}^d$ and $\omega \sim N(0, \sigma^2 I_d)$.

Another intuitive test function to try is that of polynomials. We found that empirically that these work better than the Fourier features (see below), but they have the disadvantage that they are computationally heavier, since the number of test functions of degree p is $d + d^2 + \dots + d^p$, where d is the dimension of our data. This means that we are restricted to both low-degree polynomials and low-dimension data.

Definition 3.2 (Polynomial test functions)

Suppose our dataset has dimension n , and we want to consider polynomials of degree d . The the set of polynomial test functions is

$$\{f(x) = x_i^k x_j^k | i, j \in \{0, \dots, n\}, k \in \{0, \dots, d\}\}.$$

Another is set of test functions is Hermite polynomials. We did not find much empirical improvement of regular polynomial test functions (see below).

Definition 3.3 (Hermite polynomial test functions)

$$He_n(x) = (-1)^n e^{\frac{x^2}{2}} \frac{d^n}{dx^n} e^{-\frac{x^2}{2}}$$

A key step in the algorithm 1 is that of sampling $p_t(\cdot|z)$ to generate our values x . In order to do this we need to develop our concept of conditional probability paths. The most common probability path used in generative flow models is a Gaussian probability path.

Definition 3.4 (Gaussian probability path)

Let $\alpha_t = \alpha(t) \in C^1([0, 1])$ and $\beta_t = \beta(t) \in C^1([0, 1])$ be noise schedulers, that is, monotonic functions with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. We then define the Gaussian conditional probability path to be

$$p_t(\cdot|z) = N(\alpha_t z, \beta_t^2 I_d).$$

Then this probability path fulfills the boundary conditions

$$p_0(\cdot|z) = N(\alpha_0 z, \beta_0^2 I_d) = N(0, I_d) \quad \text{and} \quad p_1(\cdot|z) = N(\alpha_1 z, \beta_1^2 I_d) = N(z, 0)$$

Since score matching only works with Gaussian probability paths, these are widely used in practice. However, Gaussian probability paths have the inherent restrictions that they force p_{simple} to also be Gaussian. Hence, if one wants to interpolate between arbitrary distributions, one must use some other probability path.

However, we found that, in practice, one is not very interested in the distribution of p_{simple} , as long as one is able to interpolate between it and p_{data} . Hence it Gaussian probability paths suffice in most cases, including ours.

3.2 Generation

Having learned our drift b_t^θ , we want to use it to generate new samples in p_{data} from our synthetic distribution p_{simple} . We can do this by simply solving the stochastic differential equation, and thus simulating its path. In our implementation we use the Euler-Maruyama solver, since it is very simple. Algorithm 2 gives the pseudocode.

Algorithm 2 Euler-Maruyama

Require: Neural net b_t^θ , number of steps n , diffusion coefficient σ_t

Set $h = \frac{1}{n}$

Set $t = 0$

Sample $X_0 \sim p_{simple}$

for $i = 1, \dots, n$ **do**

 Sample $\epsilon \sim N(0, I)$

$X_{t+h} = X_t + hb_t^\theta(X_t) + \sigma_t\sqrt{h}\epsilon$

 Update $t \leftarrow t + h$

end for

return X_1

The Euler-Maruyama method is essentially just the Euler method for deterministic differential equations, but with some simulated noise. What it does is to just take a time step in the direction predicted by our neural net b_t^θ , while adding some noise to keep it stochastic. This is the main part of the algorithm, namely updating the time step:

$$X_{t+h} = X_t + hb_t^\theta(X_t) + \sigma_t\sqrt{h}\epsilon$$

The Euler-Maruyama method converges in the following sense.

Definition 3.5

The time discrete approximation Y^δ , with maximum step size δ , converges strongly to the Itô process X with order $\gamma > 0$ at time T if there exists constant $C > 0$ and $\delta_0 > 0$ such that

$$\epsilon(\delta) = \mathbb{E} [|X_t - Y^\delta(T)|] \leq C\delta^\gamma$$

One can show that the Euler-Maruyama method converges strongly with order 0.5. [7] For more about the Euler-Maruyama method and other methods for solving SDEs, we refer the interested reader to [7]. We can now generate new samples of our p_{data} distribution, simply by sampling an element of p_{simple} , and simulating the SDE using Euler-Maruyama to get from $X_0 \sim p_{init}$ to $X_1 \sim p_{data}$.

4 Results

We implemented Alg. 1 using a few of the test functions we suggested. Since we did not see a significant difference between the regular and Hermite polynomial test functions, we present the results for regular polynomial test functions here. For our implementation, we parametrized the drift using an MLP neural network of size 4×64 . We used $\sigma = 0.5$, as well as 1000 steps and a batch size of 10000 with 5000 epochs of training. The hyperparameters were chosen simply by trial and error, while keeping computation fairly fast (at most a few minutes) on a modern CPU.

We began by learning a multiple Gaussian distribution. This is perhaps the most natural distribution for us to learn, since we are using Gaussian probability paths and we have the Gaussian noise component of the stochastic differential equation as well. For this distribution, we got very good results with all of our test functions, hence we only present the result of Fourier test functions. An example of the flow of samples and trajectories of the learned model, compared with the ground-truth probability paths, is given in Fig. 1.

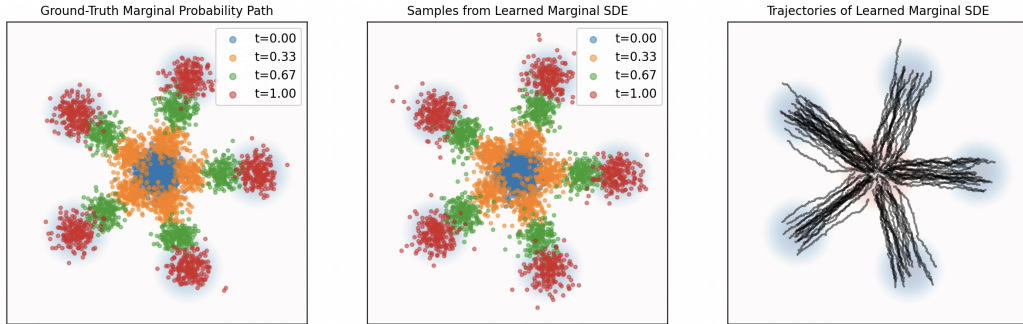


Figure 1: Learning multiple Gaussians

To distinguish between the quality of different test functions, we had to train on more difficult data distributions. However, we stuck to $2d$ distributions, since they are easy to visualize, and some problems encountered with higher dimensional data, which are discussed below. Fig. 2 shows the flow of data learned using Fourier test functions, compared to the ground-truth probability paths. Fig. 3 shows the results for learning a checkerboard data distribution and Fig. 4 the results for a circles distribution.

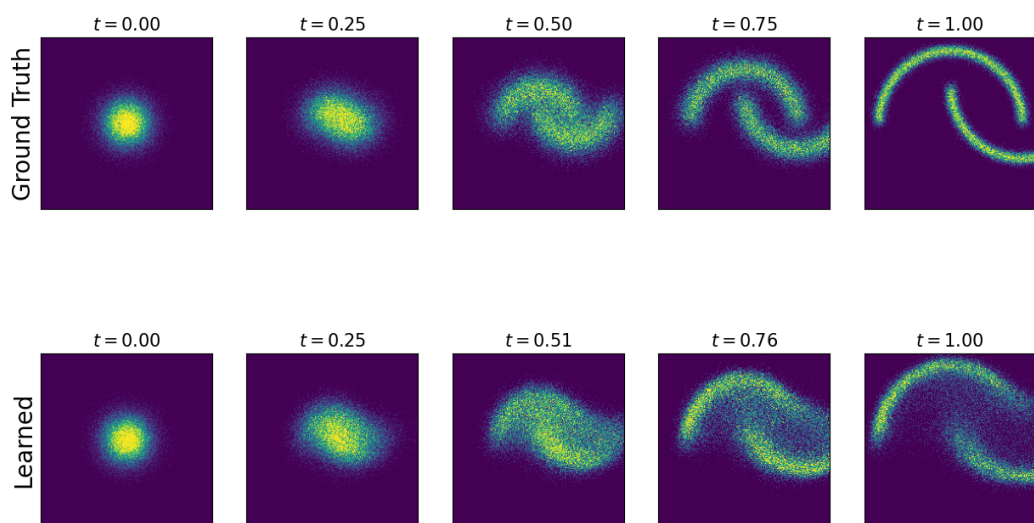


Figure 2: Learning two moons with Fourier test functions

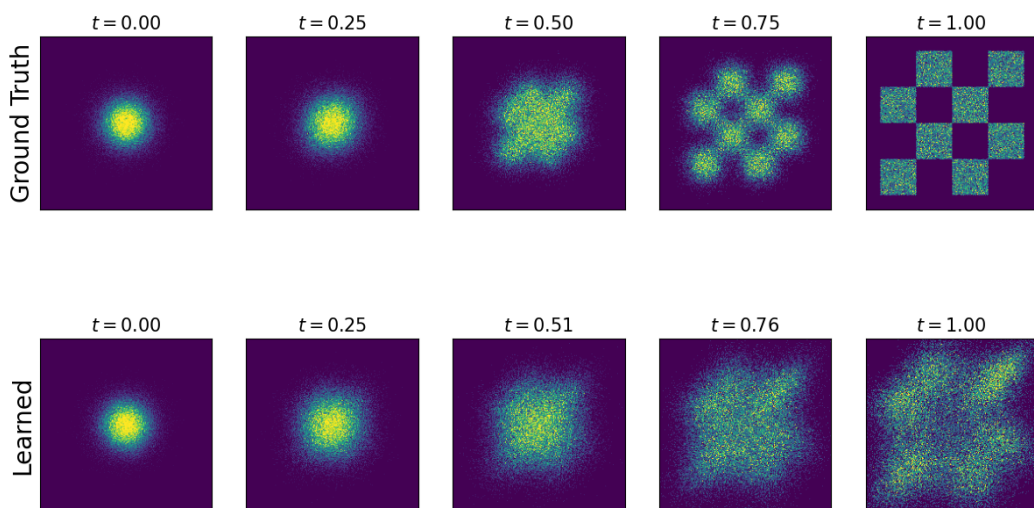


Figure 3: Learning a checkerboard with Fourier test functions

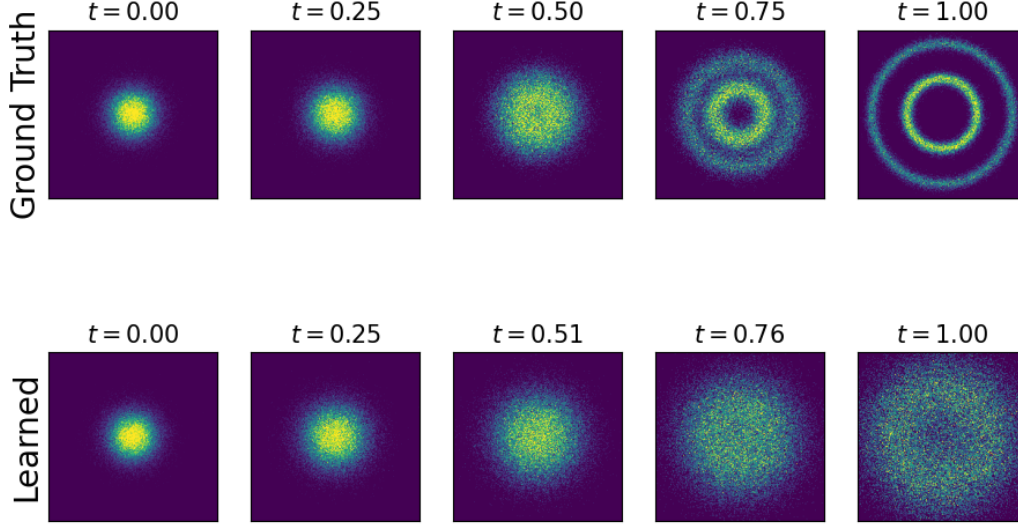


Figure 4: Learning circles with Fourier test functions

As seen in Fig. 2 and Fig. 3, the Fourier functions seem to struggle with even slightly more complicated data than variations on the Gaussian distribution. As soon as they have to learn drift which is clearly non-linear, they struggle. The Fourier test functions seem to especially struggle on the checkerboard, but here any stochastic process will slightly struggle, due to the inherent diffusion which tends to make the corners slightly fuzzy. However, the Fourier test functions seem to completely break down in Fig. 4, which shows that the problem does not lie with "sharp corners".

Hence, we then implemented polynomial test functions up to the second degree on the same data sets. Fig. 5 shows two moons and Fig. 6 shows the checkerboard, both learned using polynomial test functions. We find that the polynomial test functions do much better than the Fourier features. They are likely able to better learn higher-order terms of the drift. The polynomial test functions learn the two moons dataset almost perfectly, and the checkerboard with much less fuzziness than the Fourier test functions. Moreover, Fig. 7 shows that the polynomial test functions learn the circles dataset almost perfectly.

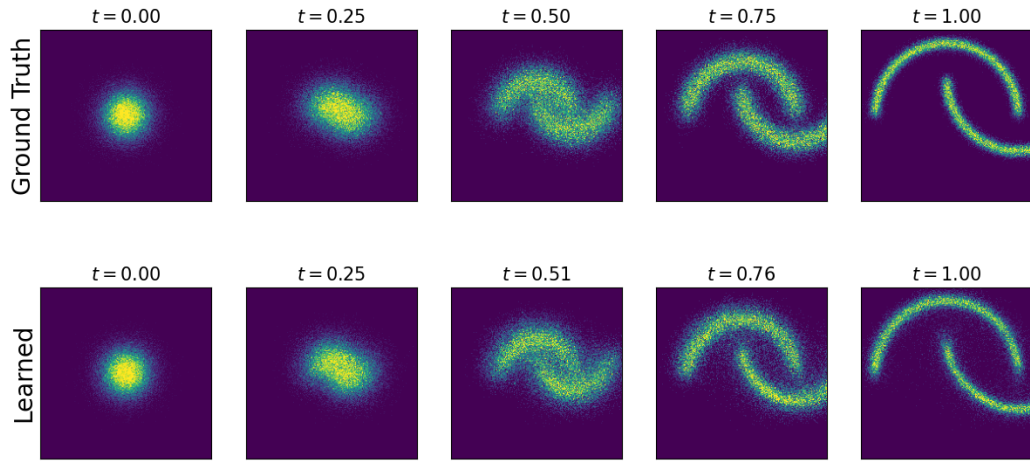


Figure 5: Learning two moons with polynomial test functions

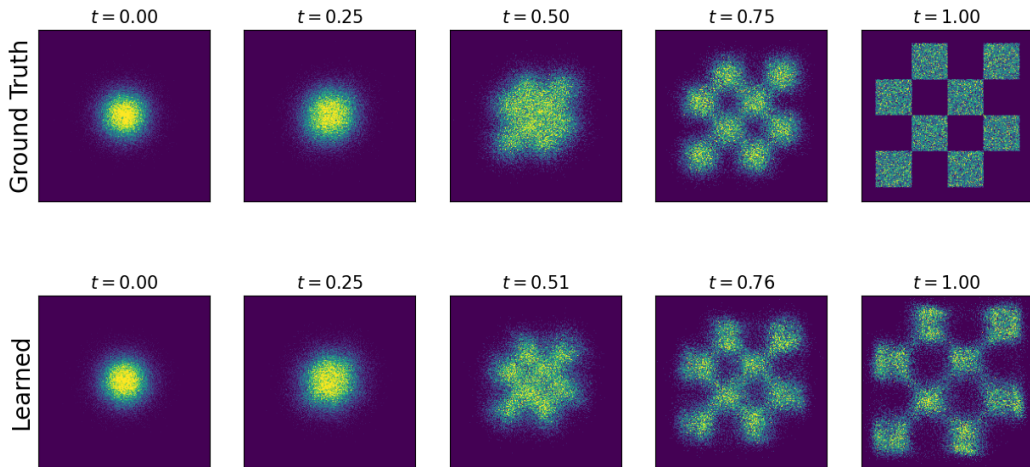


Figure 6: Learning a checkerboard with polynomial test functions

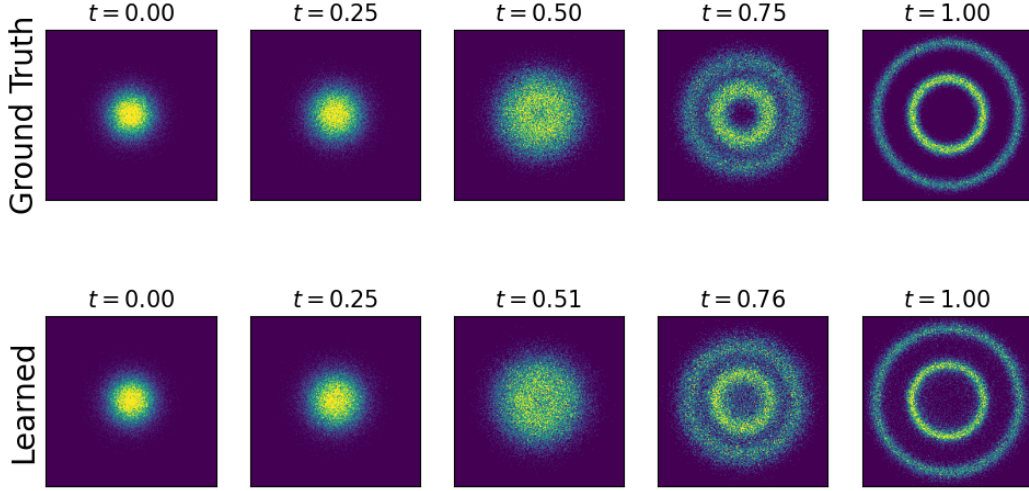


Figure 7: Learning circles with polynomial test functions

We also attempted to learn higher dimensions data, such as the MNIST dataset of handwritten digits with 28×28 pixels, which has dimension $28 \times 28 = 784$. However, we encountered the problem that our Fourier test functions were not good enough, whereas the number of our second-degree polynomial test functions grow quadratically. This means that in the MNIST case, we have $784 + 784^2 = 615440$ test functions. This significantly slows down our computation, and more importantly, requires too much memory to run on a regular computer, and so the program simply crashes.

5 Conclusion

The results we got applying martingale matching to toy data we very encouraging. Clearly we are able to learn something useful about our data simply by enforcing the martingale given in Eq. 17. The main architectural difficulty we encountered was finding test functions which give us good results while being computationally cheap.

Hence, there are a multitude of ways to extend this work, since the primary design choice for martingale matching is how we choose test functions. One option would be to look at different test functions than Fourier features or polynomials. Our choice of test functions was somewhat arbitrary, and there are surely ones that would perform significantly better. Another would

be to try to mitigate for weaknesses of different test functions by combining them in various ways, to better represent the whole space of test functions.

Perhaps the most promising extension would be to learn the test functions. If one would parametrize them by their own neural nets, then one could simply train them while training the drift. This would remove some of the design choices that have to be made while implementing the martingale matching algorithm. More importantly, it would likely decrease the number of test functions needed and significantly improve the scalability to higher dimensions. This is because a dataset like MNIST, although it has 784 pixels, in reality has many fewer true dimensions. Hence, many of the relationships captured by mixed-term polynomials likely do not contribute meaningfully to the learning process, and hence could be discarded. We hypothesize that the machine could do quite a good job at choosing the meaningful dimensions through learning.

References

- [1] Brian D.O. Anderson. “Reverse-time diffusion equation models”. In: *Stochastic Processes and their Applications* 12.3 (May 1982), pp. 313–326. DOI: 10.1016/0304-4149(82)90051-5.
- [2] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: <https://arxiv.org/abs/2006.11239>.
- [3] Peter Holderrieth and Ezra Erives. *An Introduction to Flow Matching and Diffusion Models*. 2026. arXiv: 2506.02070 [cs.LG]. URL: <https://arxiv.org/abs/2506.02070>.
- [4] Peter Holderrieth et al. *Generator Matching: Generative modeling with arbitrary Markov processes*. 2025. arXiv: 2410.20587 [cs.LG]. URL: <https://arxiv.org/abs/2410.20587>.
- [5] Olav Kallenberg. *Foundations of Modern Probability*. Springer, Jan. 2002. ISBN: 9780387953137.
- [6] Ioannis Karatzas and Steven E Shreve. *Brownian Motion and Stochastic Calculus*. New York: Springer, 1998. ISBN: 9780387976556.
- [7] Peter E Kloeden and Eckhard Platen. *Numerical Solution of Stochastic Differential Equations*. Berlin: Springer, 1999. ISBN: 9783540540625.

- [8] Yaron Lipman et al. *Flow Matching for Generative Modeling*. 2023. arXiv: 2210.02747 [cs.LG]. URL: <https://arxiv.org/abs/2210.02747>.
- [9] Yaron Lipman et al. *Flow Matching Guide and Code*. 2024. arXiv: 2412.06264 [cs.LG]. URL: <https://arxiv.org/abs/2412.06264>.
- [10] Bernt Øksendal. *Stochastic Differential Equations : An Introduction with Applications*. Heidelberg: Springer, 2013. ISBN: 9783540047582.
- [11] Jascha Sohl-Dickstein et al. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. arXiv: 1503.03585 [cs.LG]. URL: <https://arxiv.org/abs/1503.03585>.
- [12] Yang Song and Stefano Ermon. *Generative Modeling by Estimating Gradients of the Data Distribution*. 2020. arXiv: 1907.05600 [cs.LG]. URL: <https://arxiv.org/abs/1907.05600>.
- [13] Yang Song et al. *Score-Based Generative Modeling through Stochastic Differential Equations*. 2021. arXiv: 2011.13456 [cs.LG]. URL: <https://arxiv.org/abs/2011.13456>.